

RAG, explained

Retrieval-Augmented Generation: how an AI answers from **your documents** instead of from memory.

❌ Without RAG: the AI answers from memory

👤 “How many vacation days do new employees get?”

🗄️ “Typically 20 days.”

WHAT THE AI ACTUALLY KNOWS



What it learned in training

Public web pages, books and code, up to a fixed cut-off date.



Your company's documents

Handbook, policies, contracts, internal notes:
never seen.



- ⚠️ It has never read **your** handbook, so it guesses, confidently.
- ⚠️ It can't show **where** the answer came from.
- ⚠️ Its knowledge stops at the date it was trained.

✅ With RAG: the AI looks it up first



Retrieve

Search your documents for passages about the question.



Employee Handbook › Leave › “New employees receive 15 days in their first year.”



Augment

Hand the AI the question *plus* the passages found, with the instruction: use only these.



Generate

The AI writes the answer and cites its source.



“New employees get **15 days** in their first year **1**.”

Sources: **1** Employee Handbook › Leave › New employees

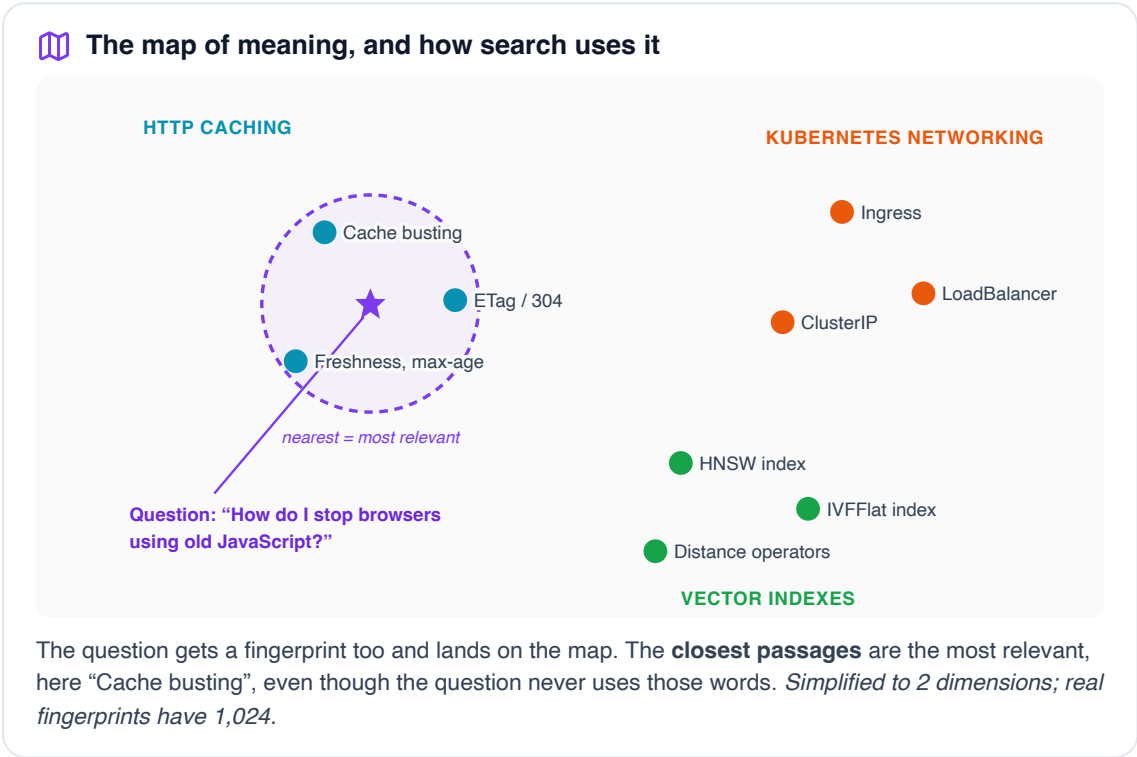
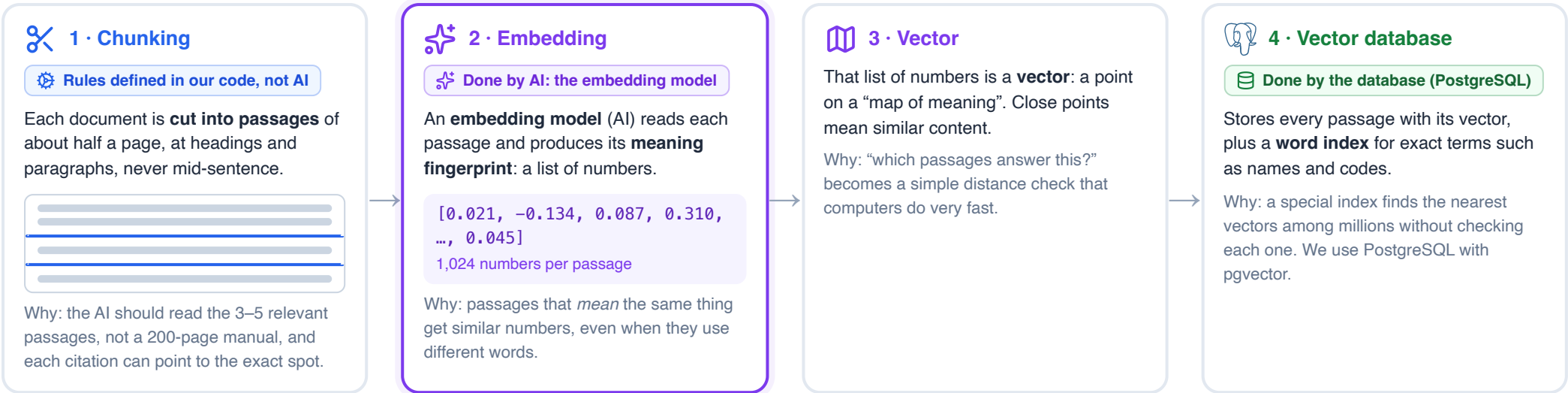
- ✅ Answers come from your own, current documents.
- ✅ Every claim links to its source, so anyone can check it.
- ✅ If the documents don't say, it answers “I couldn't find this”.



Think of an open-book exam. The AI is a bright student who must answer by looking things up in the book you gave it (your documents) and citing the page, rather than relying on what it half-remembers. Add a new document, and the “book” is up to date immediately. No retraining needed.

The building blocks

How documents are prepared so the right passage can be found in a fraction of a second.



Same idea, different names	
YOU MAY HEAR	WHAT IT MEANS
Chunk · passage	A short piece of a document.
Meaning fingerprint	Our plain-language name for an embedding.
Embedding	The numbers an embedding model produces for a text.
Vector	That list of numbers, seen as a point you can measure distances to.
Vector database	A database that stores vectors and finds the nearest ones quickly.
Semantic search	Search by meaning (nearest vectors), not by matching words.
Hybrid search	Meaning search and exact-word search combined.

How many AI models does it take?

Simple RAG and our Agentic RAG need the **same two AI models**. The difference is **what the chat model is asked to do**, and how often.



1 · Embedding model “the translator”

Turns text into meaning fingerprints. Used for **every passage at upload** and for **every question**. It is small and fast (about 0.2 s per question).

 **qwen3-embedding:8b · runs on our machine**



2 · Chat model “the reader and writer”

Reads and writes language. In simple RAG it only **writes answers**. In Agentic RAG it also **reviews** the passages found and **plans** which searches to run. This is the larger, slower model.

 **gpt-oss:20b · runs on our machine**

	Simple RAG	Agentic RAG · Ask	Agentic RAG · Agent
	 embed →  search →  write	 embed →  search →  review →  write →  check	 plan ⇄  embed  search x1–6 →  write →  check
AI models needed	2	2	2
What the chat model does	Writes the answer	Reviews the passages, then writes	Plans the searches, then writes
Searches per question	1, by meaning only	1, by meaning and exact words	1–6, chosen by the AI
AI calls per question	2	3	typically 3–5
Protection against made-up answers	Relies on the prompt alone	Irrelevant passages dropped, citations checked, “not found” when nothing matches	Citations checked; says what it searched for when nothing is found
Time per question on our machine	~4–6 s (estimate)	~8 s typical	~6 s typical, longer for complex questions
Best for	Prototypes, simple look-ups	Direct questions	Questions with several parts



Uploads use only the embedding model. The chat model is needed only when someone asks a question.



Optional third model (on our roadmap): a small, dedicated “reviewer” model could replace the chat model in the review step. Same job, about 0.2 s instead of about 4 s.



Why not one model for everything? Each model does a different job. Embedding models compare meaning quickly, covering thousands of passages per minute; chat models read and write. Searching with a chat model would be far too slow, and an embedding model cannot write an answer.

Ask vs Agent, in detail

Both modes are in our app (the **Ask | Agent** switch) and use the same two AI models. The difference: **who decides what to search for**.

 **Example question with two parts:** “How do users on the internet reach my HTTPS API on Kubernetes, and how do I stop browsers from using an outdated JavaScript bundle?”



Ask

A fixed pipeline: **our code** decides the steps

1**Fingerprint the question**

AI · embedding

The whole question is used as-is, as one search.

2**Search once**

database

By meaning and exact words; the best 10 passages come back.

3**Review**

AI · chat

The chat AI grades each passage 0–3 and drops the irrelevant ones.

4**Write**

AI · chat

The answer is written from the kept passages only, citing [n].

5**Check**

code

Every [n] must point to a real passage.

For the example, **one search** must cover *both* topics. It often works, because hybrid search returns passages about Kubernetes and caching, but one part can be missed when the question mixes topics.

Always 1 search

3 AI calls

~8 s typical



Agent

A loop: **the AI** decides what to search, and when to stop

1**Plan**

AI · chat

The chat AI reads the question and decides what to look up first.

repeats 1–6 times

2**Search with its own words**

AI · embedding + database

The AI writes a short, focused query; it is fingerprinted and searched, and 5 numbered passages come back.

3**Read and decide**

AI · chat

Enough to answer? If not, search again with different words, or read around a passage.

4**Write**

AI · chat

The answer is written from everything it found, citing [n].

5**Check**

code

Every [n] must point to a passage it actually read.

A REAL RUN OF THE EXAMPLE QUESTION

search 1: “internet reach HTTPS API on Kubernetes” → 5 passages (*Ingress, LoadBalancer...*)

search 2: “stop browsers from using an outdated JavaScript bundle cache busting” → 5 passages

an answer covering both parts, citing 5 sources · 2 searches · 18.5 s



What “x1–6” means

The search step can repeat **up to 6 times**, a safety limit so the AI can’t loop forever. Reading around a passage or listing the documents also counts. At the limit, the tools are taken away and the AI must answer with what it has. In our evaluation the agent needed **1.3 searches** per question on average.



Which one should I use?

Direct question with clear words (“ <i>What is the default ef_search?</i> ”)	Ask
Two or more parts, or a comparison (“ <i>Compare HNSW and IVFFlat</i> ”)	Agent
Unsure which words the documents use	Agent
Fastest, most predictable result	Ask



Trade-offs

	Ask	Agent
Searches	1	1–6
Quality review step	yes	AI judges itself
Tokens per question*	~575	~2,500
Predictability	high	varies

*From our evaluation. More tokens means more cost with paid AI services.

Agentic RAG · RAG explained · Ask vs Agent in detail

Appendix